Lesson: Selection Statements

CSE 4107 : Structured Programming I

Shahriar Ivan

Statements

- So far, we've used return statements and expression statements.
- Most of C's remaining statements fall into three categories:
  - ***Selection statements:*** *if* and *switch*
  - ***Iteration statements:*** *while*, *do*, and *for*
  - ***Jump statements:*** *break*, *continue*, and *goto*. (*return* also belongs in this category.)
- Other C statements:
  - Compound statement
  - Null statement

Logical Statements

Purpose

- Test the value of an expression to see if it “true” or “false”
- Example:
  - if statement -> i < j?
  - A true value would indicate that i is less than j
- Other programming languages an expression such as i < j would have a special “Boolean” or “Logical” type
- But in C, such comparison statements yields an integer: 0 or 1

Logical Statements

Relational Operators

- C's ***relational operators:***

<	less than
>	greater than
<=	less than or equal to
>=	greater than or equal to

- These operators produce `0` (false) or `1` (true) when used in expressions
- The relational operators can be used to compare integers and floating-point numbers, with operands of mixed types allowed

Logical Statements

Relational Operators

- The precedence of the relational operators is lower than that of the arithmetic operators
    - For example, `i + j < k - 1` means `(i + j) < (k - 1)`
- The relational operators are left associative

Logical Statements

Relational Operators

- The expression
  - `i < j < k`
  - is legal, but does not test whether `j` lies between `i` and `k`
- Since the `<` operator is left associative, this expression is equivalent to

  `(i < j) < k`
- The `1` or `0` produced by `i < j` is then compared to `k`
- The correct expression is `i < j && j < k`

Logical Statements

Equality Operators

- C provides two ***equality operators:***
  - `==` equal to
  - `!=` not equal to
- The equality operators are left associative and produce either `0` (false) or `1` (true) as their result
- The equality operators have lower precedence than the relational operators, so the expression
  - `i < j == j < k` is equivalent to, `(i < j) == (j < k)`

Logical Statements

Logical Operators

- More complicated logical expressions can be built from simpler ones by using the ***logical operators:***
  - **!** logical *negation*
  - **&&** logical *and*
  - **||** logical *or*
- The **!** operator is unary, while **&&** and **||** are binary.
- The logical operators produce 0 or 1 as their result.
- The logical operators treat any nonzero operand as a true value and any zero operand as a false value

Logical Statements

Logical Operators

- Behavior of the logical operators:
  - !*expr* has the value 1 if *expr* has the value 0
  - *expr1* && *expr2* has the value 1 if the values of *expr1* and *expr2* are both nonzero
  - *expr1* || *expr2* has the value 1 if either *expr1* or *expr2* (or both) has a nonzero value
- In all other cases, these operators produce the value 0

Logical Statements

Logical Operators

- Both && and || perform "*short-circuit*" evaluation: they first evaluate the left operand, then the right one
- If the value of the expression can be deduced from the left operand alone, the right operand isn't evaluated
- Example:
  - (i != 0) && (j / i > 0)
  - (i != 0) is evaluated first. If i isn't equal to 0, then (j / i > 0) is evaluated
- If i is 0, the entire expression must be false, so there's no need to evaluate (j / i > 0). Without short-circuit evaluation, division by zero would have occurred

Logical Statements

Logical Operators

- Side effects in logical expressions may not always occur because of the *short-circuit* nature
- Example:
  - `i > 0 && ++j > 0`
- If `i > 0` is false, then `++j > 0` is not evaluated, so j isn't incremented
- The problem can be fixed by changing the condition to `++j > 0 && i > 0` or, even better, by incrementing j separately

Logical Statements

Logical Operators

- The ! operator has the same precedence as the unary plus and minus operators
- The precedence of && and || is lower than that of the relational and equality operators
  - For example, `i < j && k == m` means `(i < j) && (k == m)`
- The ! operator is right associative; && and || are left associative

The if Statement

- The if statement allows a program to choose between two alternatives by testing an expression
- In its simplest form, the if statement has the form:

```
if (expression) statement
```

- When an if statement is executed, ***expression*** is evaluated; If its value is nonzero, ***statement*** is executed
- Example:

```
if (line_num == MAX_LINES)
    line_num = 0;
```

The if Statement

- Confusing == (equality) with = (assignment) is perhaps the most common C programming error
- The statement

```
if (i == 0) ...
```

  tests whether i is equal to 0

- The statement

```
if (i = 0) ...
```

  assigns 0 to i, then tests whether the result is nonzero

The if Statement

Compound Statements

- To make an if statement control two or more statements, use a ***compound statement***
- A compound statement has the form

  { *statements* }

- Example:

```
{
        line_num = 0;
        page_num++;
}
```

The if Statement

Compound Statements

- A compound statement is usually put on multiple lines, with one statement per line:

```
{
        line_num = 0;
        page_num++;
}
```

- Each inner statement still ends with a semicolon, but the compound statement itself does not

The if Statement

The else Clause

- An if statement may have an else clause:

  if (*expression*) *statement* else *statement*

- The statement that follows the word *else* is executed if the expression has the value 0
- Example:

```
if (i > j)
        max = i;
else
        max = j;
```

```
if (i > j)    max = i;
else  max= j;
```

The if Statement

Nested if Statement

```
// Indentation makes it much more readable
if (i > j)
        if (i > k)
                max = i;
        else
                max = k;
else
        if (j > k)
                max = j;
        else
                max = k;
```

```
// Can use braces too
if (i > j) {
        if (i > k)
                max = i;
        else
                max = k;
}
else {
        if (j > k)
                max = j;
        else
                max = k;
}
```

The if Statement

Cascaded if Statement

- A “cascaded” if statement is often the best way to test a series of conditions, stopping as soon as one of them is true.
- Example:

```
if (n < 0)
        printf("n is less than 0\n");
else
        if (n == 0)
                printf("n is equal to 0\n");
        else
                printf("n is greater than 0\n");
```

The if Statement

Cascaded if Statement

- Although the second if statement is nested inside the first, C programmers don't usually indent it.
- Instead, they align each else with the original if:

```
if (n < 0)
    printf("n is less than 0\n");
else if (n == 0)
    printf("n is equal to 0\n");
else
    printf("n is greater than 0\n");
```

Programming Task

Calculating a Broker's Commission

- When stocks are sold or purchased through a broker, the broker's commission often depends upon the value of the stocks traded
- Suppose that a broker charges the amounts shown in the following table:

Transaction size	*Commission rate*
Under $2,500	$30 + 1.7%
$2,500–$6,250	$56 + 0.66%
$6,250–$20,000	$76 + 0.34%
$20,000–$50,000	$100 + 0.22%
$50,000–$500,000	$155 + 0.11%
Over $500,000	$255 + 0.09%

- The minimum charge is $39

Programming Task

Calculating a Broker's Commission

- The program asks the user to enter the amount of the trade, then displays the amount of the commission:

  Enter value of trade: 30000

  Commission: $166.00

- The heart of the program is a cascaded if statement that determines which range the trade falls into

The if Statement

The "Dangling else" Problem

- When if statements are nested, the "dangling else" problem may occur:

```
if (y != 0)
        if (x != 0)
                result = x / y;
else
        printf("Error: y is equal to 0\n");
```

- The indentation suggests that the else clause belongs to the outer if statement
- However, C follows the rule that an else clause belongs to the nearest if statement that hasn't already been paired with an else

The if Statement

The “Dangling else” Problem

- A correctly indented version would look like this:

```
if (y != 0)
        if (x != 0)
                result = x / y;
        else
                printf("Error: y is equal to 0\n");
```

The if Statement

The “Dangling else” Problem

- A correctly indented version would look like this:

```
if (y != 0)
  if (x != 0)
    result = x / y;
  else
    printf("Error: y is equal to 0\n");
```

```
if (y != 0)
   if (x != 0)
     result = x / y;
   else
     printf("Error: x is equal to 0\n");
else
   printf("Error: y is equal to 0\n");
```

Conditional Expressions

- C's ***conditional operator*** allows an expression to produce one of two values depending on the value of a condition
- The conditional operator consists of two symbols (`?` and `:`), which must be used together:

  expr1 `?` *expr2* `:` *expr3*

- The operands can be of any type
- The resulting expression is said to be a ***conditional expression***

Conditional Expressions

- The conditional operator requires three operands, so it is often referred to as a ***ternary*** operator
- The conditional expression *expr1* ? *expr2* : *expr3* should be read "if *expr1* then *expr2* else *expr3*."
- The expression is evaluated in stages: *expr1* is evaluated first; if its value isn't zero, then *expr2* is evaluated, and its value is the value of the entire conditional expression.
- If the value of *expr1* is zero, then the value of *expr3* is the value of the conditional

Conditional Expressions

- Example:

```
int i, j, k;
i = 1;
j = 2;
k = i > j ? i : j;              /* k is now 2 */
k = (i >= 0 ? i : 0) + j;   /* k is now 3 */
```

- The parentheses are necessary, because the precedence of the conditional operator is less than that of the other operators discussed so far, with the exception of the assignment operators

Conditional Expressions

- Makes program shorter, but harder to understand
- Conditional expressions are often used in return statements:

```
return i > j ? i : j;
```

Conditional Expressions

- Calls of printf can sometimes benefit from condition expressions. Instead of

```
if (i > j)
        printf("%d\n", i);
else
        printf("%d\n", j);
```

we could simply write

```
        printf("%d\n", i > j ? i : j);
```

- Conditional expressions are also common in certain kinds of macro definitions

The switch Statement

- A cascaded if statement can be used to compare an expression against a series of values:

```
if (grade == 3)
        printf("Excellent");
else if (grade == 2)
        printf("Average");
else if (grade == 1)
        printf("Poor");
else if (grade == 0)
        printf("Failing");
else
        printf("Illegal grade");
```

The switch Statement

- The switch statement is an alternative:

```
switch (grade) {
        case 3:  printf("Excellent");
                 break;
        case 2:  printf("Average");
                 break;
        case 1:  printf("Poor");
                 break;
        case 0:  printf("Failing");
                 break;
        default: printf("Illegal grade");
                 break;
}
```

The switch Statement

- A switch statement may be easier to read than a cascaded if statement.
- switch statements are often faster than if statements.
- Most common form of the switch statement:

```
switch ( expression ) {
        case constant-expression : statements
        ...
        case constant-expression : statements
        default : statements
}
```

The switch Statement

- The word switch must be followed by an integer expression—the ***controlling expression***—in parentheses
- Characters are treated as integers in C and thus can be tested in switch statements
- However, floating-point numbers and strings don’t qualify

The switch Statement

- Each case begins with a label of the form

  case *constant-expression* :

- A ***constant expression*** is much like an ordinary expression except that it can't contain variables or function calls

  – `5` is a constant expression, and `5 + 10` is a constant expression, but `n + 10` isn't a constant expression (unless n is a macro that represents a constant)

- The constant expression in a case label must evaluate to an integer (characters are acceptable)

The switch Statement

- After each `case` label comes any number of statements
- No braces are required around the statements
- The last statement in each group is normally `break`

The switch Statement

- Duplicate case labels aren't allowed and the order of cases doesn't matter
- Default case doesn't need to come last
- Several case labels may precede a group of statements:

```
switch (grade) {
      case 3:
      case 2:
      case 1:    printf("Passing");
                 break;
      case 0:    printf("Failing");
                 break;
      default:   printf("Illegal grade");
                 break;
}
```

The switch Statement

- To save space, several case labels can be put on the same line:

```
switch (grade) {
        case 3: case 2: case 1:
                printf("Passing");
                break;
        case 0:  printf("Failing");
                break;
        default:  printf("Illegal grade");
                break;
}
```

- If the default case is missing and the controlling expression is false, control passes to the next statement after the switch

The Role of the `break` Statement

- Executing a `break` statement causes the program to "`break`" out of the `switch` statement; execution continues at the next statement after the `switch`
- The `switch` statement is really a form of "*computed jump*"
- When the controlling expression is evaluated, control jumps to the `case` label matching the value of the `switch` expression.
- A `case` label is nothing more than a marker indicating a position within the `switch`

The Role of the break Statement

- Without break (or some jump statement), control will flow into the next case
- Example:

```
switch (grade) {
        case 3:  printf("Excellent");
        case 2:  printf("Average");
        case 1:  printf("Poor");
        case 0:  printf("Failing");
        default: printf("Illegal grade");
}
```

- If the value of grade is 3, the message printed is

  "ExcellentAveragePoorFailingIllegal grade"

The Role of the break Statement

- Omitting break is sometimes done intentionally, but it's usually just an oversight.
- It's a good idea to point out deliberate omissions of break:

```
switch (grade) {
  case 4: case 3: case 2: case 1:
        num_passing++;
        /* FALL THROUGH */
  case 0: total_grades++;
        break;
}
```

Programming Task

Printing a Date in Legal Form

- Contracts and other legal documents are often dated in the following way:

  Dated this _____ day of _____ , 20__

- The program will display a date in this form after the user enters the date in month/day/year form:

  Enter date (mm/dd/yy): 7/19/14

  Dated this 19th day of July, 2014

- The program uses switch statements to add “th” (or “st” or “nd” or “rd”) to the day, and to print the month as a word instead of a number